



윤경은

Wrtn Agent Developer Intern

yge0307@gmail.com
github.com/ykgstar37-lab
yge-portfolio.vercel.app

Agent 실패를 측정하고, 검증 구조로 바로잡는 Python 백엔드/AI 엔지니어

핵심역량

- **Agent 신뢰성 설계** — 팀 4명 프로젝트에서 sLLM 과신 응답을 검증 구조 문제로 재정의하고, 4중 Guardrail + Confidence 보정으로 정확도 **37%→85%**, JSON 유효율 **70%→97%** 개선 (WorkFlow Agent)
- **RAG 병목 진단** — 팀 4명 프로젝트에서 RAGAS로 검색·생성을 분리 측정해 LLM이 아닌 embedding 병목을 확인하고, Context Precision **0.83→0.97**, Recall **0.70→0.79** 개선 (PyMate)
- **Agent 비용·라우팅 설계** — 개인 프로젝트에서 Intent→Retrieve→Generate 3-node 라우팅을 설계해 요청당 비용 **\$0.003**, 단일 LLM 호출 대비 **85% 절감** 달성 (Seoul Culture Map)
- **Python 백엔드 end-to-end** — 최근 7개월간 3개 프로젝트에서 FastAPI/Django 서버, **11개 REST API**, SSE, Docker 서버, AWS EC2 배포, Nginx·Gunicorn **502 장애 해결**까지 직접 수행

EDUCATION

가천대학교 응용통계학과

2020.03 ~ 2026.08 (졸업 예정)

SK Networks Family AI Camp

2025.09 ~ 2026.03 · AI/ML 엔지니어 집중 과정 수료

CERTIFICATES

SQLD

한국데이터산업진흥원

ADsP

한국데이터산업진흥원

SKILLS

AGENT / LLM

LangGraph Guardrail OpenAI API vLLM LoRA RAGAS Qdrant ChromaDB HyDE BM25+RRF

BACKEND / INFRA

Python FastAPI Django PostgreSQL SQLite

Docker Nginx AWS EC2 SSE

DATA / STATISTICS

pandas scikit-learn numpy GARCH K-means

PCA SQL

SELECTED IMPACT

AGENT RELIABILITY WORKFLOW AGENT

37% → 85%

**4중 Guardrail
+ Confidence 보정**

존재하지 않는 조항을 근거처럼 말하는 sLLM 응답을 서비스 레이어에서 먼저 차단했습니다.

RAG QUALITY PYMATE

0.83 → 0.97

RAGAS 기반 병목 진단

LLM 교체보다 먼저 검색과 생성을 분리 측정해 실제 병목을 확인했습니다.

COST OPTIMIZATION SEOUL CULTURE MAP

85% ↓

3-node Intent Routing

검색이 필요 없는 요청은 건너뛰도록 라우팅해 비용 곡선을 낮췄습니다.

PROJECTS

WorkFlow Agent · 사내 업무 보조용 Agent 시스템

Team 4명 · 2026.02 ~ 04 · 8주

GitHub · 내 역할: 판단 Agent, Guardrail, LoRA 데이터셋, vLLM 서버 | 기여도 65%

팀 4명 기준, 판단 Agent·Guardrail·LoRA·서버 구조를 맡은 범위입니다.

sLLM이 존재하지 않는 규정을 근거처럼 말하면서 confidence **0.92**를 주는 상황을 **검증 구조 문제로** 재정의한 프로젝트입니다.

문제: 규정 판단 도메인에서 오답·환각·과신이 함께 발생해, 고신뢰 오답이 서비스 레이어를 통과할 위험이 있었습니다.

해결: 판단 Agent에 **4중 Guardrail**과 **Confidence 보정**을 넣고, GPT/Claude/vLLM을 provider 패턴으로 교체 가능하게 통일했습니다.

결과: LoRA **v1→v3 3회 이터레이션**으로 정확도 **37%→85%**, **JSON 유효율 70%→97%**를 만들고, 과신 confidence를 **0.92→0.78**로 보정했습니다.

배운 점: v2에서 **98건** 추가 학습 후 정확도 **-3.2%p** 하락을 확인하고, 오답 로그 **19건**만 다시 설계해 v3로 수정했습니다. Agent 신뢰성은 모델 성능보다 실패 처리 구조까지 함께 설계해야 안정화된다는 기준을 얻었습니다.

LangGraph vLLM LoRA Guardrail FastAPI Qdrant Docker RunPod A100

Seoul Culture Map · 서울 문화행사 Intent 라우팅 챗봇

Personal · 2026.03 ~ Present · Prototype

GitHub · 개인 프로젝트 · 설계·구현 전 과정 단독 진행 | 기여도 100%

문제 정의부터 라우팅 구조, API, UI, 실행 환경 구성까지 단독으로 진행한 프로젝트입니다.

PDF로 묻히던 분석 결과를 서울시 문화시설 **2,500+**개 탐색·추천 서비스로 전환하고, 모든 요청을 단일 LLM 호출에 태우는 비용 문제를 **Intent 라우팅**으로 해결했습니다.

문제: 분석 결과가 문서로만 남아 재사용되지 못했고, 단순 인사와 실제 검색 질의가 같은 LLM 파이프라인을 타며 비용이 커졌습니다.

해결: **Intent→Retrieve→Generate 3-node**로 분리해 일상 대화는 검색을 스킵하고, 검색형 요청만 SQL+ChromaDB RAG로 보내도록 설계했습니다.

결과: 로컬 임베딩과 SSE 스트리밍까지 적용해 요청당 비용 **\$0.003**, 단일 LLM 호출 대비 **85% 절감**, 첫 토큰 **~500ms**, 세션 문맥 **10턴**을 만들었습니다.

배운 점: Agent routing은 "언제 LLM을 호출할지"보다 "언제 호출하지 않을지"를 고르는 비용 설계 문제라는 기준을 얻었습니다.

OpenAI API LangGraph ChromaDB FastAPI SSE SQLite React

PyMate · RAG 학습 서포터 챗봇

Team 4명 · 2026.01 ~ 02 · 6주

GitHub · 내 역할: RAG 평가, 임베딩 교체, Flask→Django 전환, 배포 담당 | 기여도 70%

팀 4명 기준, 검색 품질 진단·임베딩 교체·프로덕션 전환을 맡은 범위입니다.

"답변이 부정확하다"는 증상을 LLM 탓으로 돌리지 않고, RAGAS로 검색·생성을 분리 측정해 **embedding 병목**으로 재정의한 프로젝트입니다.

문제: 검색과 생성 단계가 섞여 있어, 더 비싼 LLM으로 바뀌도 실제 원인을 놓칠 위험이 있었습니다.

해결: RAGAS로 **Context Precision 0.83** 병목을 확인한 뒤, 임베딩을 **768D→3072D**로 교체하고 HyDE+BM25+RRF+Reranker 9단계 검색 구조를 재설계했습니다.

결과: **Precision 0.83→0.97, Recall 0.70→0.79**를 만들고, Flask MVP를 Django로 전환한 뒤 AWS EC2의 Nginx·Gunicorn **502 장애**까지 해결했습니다.

배운 점: "측정 먼저, 개선은 그 다음"이라는 기준이 WorkFlow Agent의 LoRA 약점 타겟팅 판단까지 이어졌습니다.

RAGAS Qdrant Django Nginx AWS EC2

STATISTICS FOUNDATION

측정 기준 먼저 설계하는 습관

응용통계학 전공을 통해 "측정이 먼저, 개선은 그 다음"이라는 기준을 익혔고, 최근 7개월간 3개 프로젝트에서 정확도, Precision/Recall, JSON validity, confidence, 비용, latency **6종 지표**를 먼저 정의한 뒤 구조를 바꿨습니다.

실험 결과 해석과 다음 액션 연결

GARCH, 회귀, 군집분석, PCA 경험을 바탕으로 수치 변화를 해석하고, PyMate의 RAG 품질 개선과 WorkFlow Agent의 LoRA **3회 이터레이션**에서 실패한 실험을 다음 판단 기준으로 바꿨습니다.

WHY WRTN AX CIC

왜 워튼인가

워튼 AX CIC 공고의 핵심을 **Productivity Bottleneck**을 정의하고, 검증 가능한 **Agentic AI**로 바꾸는 일로 이해했습니다. 저는 WorkFlow Agent에서 과신 오답을 검증 구조로, PyMate에서 "모델 문제"를 검색 병목으로, Seoul Culture Map에서 단일 호출 구조를 3-node 라우팅으로 다시 정의해 왔습니다.

강점이 직무에 닿는 방식

LangGraph Agent, RAGAS 평가, Guardrail 설계, FastAPI/Django 백엔드, vLLM/LoRA **3회 이터레이션**, AWS 배포·장애 대응 경험은 워튼이 요구하는 **Python 기반 Agent 개발 + 빠른 실험 + 검증 가능한 개선**과 맞닿아 있습니다.